

AAP-57833

This document contains only the content that was updated or created - not the entire guide.

Review tracker

 Reviewer	 Status	 Notes
Rohit Thakur	Not started	
Dafne Mendoza	Done	Jklech applied Dafne's feedback

Technical Verification Summary

Sources consulted:

- Jira ticket AAP-57833 and parent initiative ANSTRAT-1640
- ansible.platform collection source code (v2.7.20260313) — CHANGELOG, module source, doc_fragments
- Existing Configuration as Code guide in aap-docs
- Related tickets: AAP-66424 (overlap), AAP-66503, AAP-65103 (ansible.hub removals)

Acceptance criteria coverage: 2/3

- [x] CaC product guide updated with latest ansible.platform changes
- [x] Migration guidance section documenting what needs to change for 2.7
- [] Release notes — not included in this review (separate deliverable)

Verification results: 27/28 technical claims verified from source code

Items for SME confirmation:

- The feature_flag module defaults to exists instead of present — verified from source, but please confirm this is intentional and should be documented as a caveat.
- The authenticator module OIDC User Identity support — verified from CHANGELOG but could not inspect the full parameter set. Please confirm the description is accurate.
- The enable_mtls and enable_gateway_auth parameters are described as mutually exclusive — verified from source code validation logic. Please confirm this is the intended behavior.

Chapter 1. Configure your platform with Configuration as Code

1.1 Configuration as Code with the `ansible.platform` collection

(New section)

Configuration as Code (CaC) is an approach where you define and manage the configuration of Ansible Automation Platform using version-controlled files, such as YAML, instead of manually configuring resources through the web UI.

The `ansible.platform` collection provides Ansible modules for managing Ansible Automation Platform resources programmatically.

You can use this collection to automate the creation and management of organizations, users, teams, role-based access control (RBAC), authentication providers, gateway services, and platform-wide settings.

Benefits of Configuration as Code

- **Repeatability:** Apply the same configuration across development, staging, and production environments.
- **Change tracking:** Store configuration in Git to maintain a history of changes with diffs and rollback capability.
- **Disaster recovery:** Rebuild your Ansible Automation Platform configuration quickly after outages or migrations.
- **Reduced errors:** Route changes through CI/CD pipelines and pull requests for peer review and automated testing.
- **Scalability:** Configure multiple organizations, teams, and users in bulk without manual intervention.

The four-state model

Most modules in the `ansible.platform` collection support a four-state model for managing resources:

- **present** — Create the resource if it does not exist, or update it if it does. Only the parameters you specify are set; other fields retain their current values.
- **absent** — Delete the resource if it exists.
- **exists** — Check whether the resource exists without creating or modifying it. Use this state to verify that prerequisites are in place before running further tasks.
- **enforced** — Create or update the resource, and reset any parameters you did not specify to their API default values. Use this state when you want to ensure the complete desired state of a resource, not just the fields you explicitly define.

The following example uses the enforced state to ensure that an organization has only the specified description. Any other fields that were previously set on this organization are reset to their default values:

```
- name: Enforce organization configuration
  ansible.platform.organization:
    name: "Production"
    description: "Production automation resources"
    state: enforced
```

NOTE: Not all modules support every state. The settings module has no state parameter and always applies changes. The token module supports only present and absent and is not idempotent. The feature_flag module defaults to exists instead of present, so you must explicitly set state: present to modify a flag. For a complete list of supported states per module, see the module reference.

Resource names instead of IDs

When you reference resources such as organizations, teams, or users in your playbooks, use human-readable names instead of numeric IDs. Names are portable across environments, which means you can apply the same playbook to development, staging, and production without modification. Using names also simplifies disaster recovery because you can rebuild your configuration without looking up IDs from a previous environment.

Check mode and idempotency

All resource modules in the ansible.platform collection support Ansible check mode (--check). In check mode, modules report what changes they would make without applying them. Use check mode to verify your playbooks before applying changes to a production environment.

Most modules are idempotent: running the same playbook multiple times produces the same result. The token module is an exception — each run creates a new token.

Module defaults for connection parameters

The ansible.platform collection defines an action group called gateway that includes all modules. Use module_defaults to set connection parameters once instead of repeating them in every task:

```
- name: Configure Ansible Automation Platform resources
  hosts: localhost
  gather_facts: false
  module_defaults:
    group/ansible.platform.gateway:
      aap_hostname: "{{ aap_hostname }}"
      aap_username: "{{ aap_username }}"
      aap_password: "{{ aap_password }}"
      aap_validate_certs: "{{ aap_validate_certs }}"
  tasks:
    - name: Ensure organization exists
      ansible.platform.organization:
        name: "My Organization"
```

```
state: present
```

```
- name: Ensure team exists
  ansible.platform.team:
    name: "My Team"
    organization: "My Organization"
    state: present
```

You can also use environment variables instead of specifying connection parameters in your playbook. The following environment variables are supported:

- AAP_HOSTNAME or GATEWAY_HOSTNAME
- AAP_USERNAME or GATEWAY_USERNAME
- AAP_PASSWORD or GATEWAY_PASSWORD
- AAP_TOKEN or GATEWAY_API_TOKEN
- AAP_VALIDATE_CERTS or GATEWAY_VERIFY_SSL
- AAP_REQUEST_TIMEOUT or GATEWAY_REQUEST_TIMEOUT

Note: The recommended naming convention for environment variable names is to use AAP_.*.

Authentication methods

You can authenticate to Ansible Automation Platform by using one of the following methods:

- **Username and password:** Set aap_username and aap_password.
- **OAuth2 token:** Set aap_token with a token string or a token object returned by the ansible.platform.token module.

All authentication parameters accept both aap_ and gateway_ prefixed names for backward compatibility, but the recommended option is to use aap_ prefix.

Additional resources

- Modules in the ansible.platform collection (see section 1.3 below)
- Set up your automation environment for Configuration as Code (see section 1.2 below)
- ansible.platform collection on Automation hub
(<https://console.redhat.com/ansible/automation-hub/repo/published/ansible/platform/content/?showing=module>)

1.2 Set up your automation environment for Configuration as Code

(Existing section — only the title was changed and one URL was fixed)

Title change: Changed from gerund form "Setting up your automation environment for Configuration as Code" to imperative form "Set up your automation environment for Configuration as Code" to align with Red Hat SSG title standards.

URL fix: Updated the execution environment documentation link to point to the correct AAP 2.7 URL with the {PlatformVers} attribute.

The body content of this procedure is unchanged and is not included in this review.

1.3 Modules in the ansible.platform collection

(New section)

The ansible.platform collection provides modules for managing Ansible Automation Platform resources. The following tables list the available modules grouped by category.

Identity and access management

Module	Description	Supported states
organization	Create, update, or delete organizations.	present, absent, exists, enforced
user	Create, update, or delete users. Configure superuser status and authenticator associations.	present, absent, exists, enforced
team	Create, update, or delete teams within an organization.	present, absent, exists, enforced
role_definition	Create, update, or delete custom RBAC role definitions with specific permissions.	present, absent, exists, enforced
role_user_assignment	Assign roles to users for specific resources or organizations.	present, absent, exists
role_team_assignment	Assign roles to teams for specific resources or organizations. Supports batch operations with assignment_objects.	present, absent, exists

Authentication

Module	Description	Supported states
authenticator	Configure authentication providers such as LDAP, OIDC, SAML, and GitHub.	present, absent, exists, enforced
authenticator_map	Define authentication mapping rules to map external groups to Ansible Automation Platform roles, teams, and organizations.	present, absent, exists, enforced
authenticator_user	Associate users with authentication providers for migration between providers. This module does not support deleting associations.	present, exists

Gateway infrastructure

Module	Description	Supported states
service	Configure API service routes for automation controller, automation hub, and Event-Driven Ansible controller.	present, absent, exists, enforced
route	Configure non-API gateway routes.	present, absent, exists, enforced
service_cluster	Manage service clusters with health check and outlier detection settings.	present, absent, exists, enforced
service_node	Add or remove individual service nodes within clusters.	present, absent, exists, enforced

Module	Description	Supported states
service_type	Define service type definitions with login and logout paths.	present, absent, exists, enforced
service_key	Manage service authentication keys for inter-service communication.	present, absent, exists, enforced
http_port	Configure HTTP listener ports for the Envoy proxy.	present, absent, exists, enforced
ui_plugin_route	Configure UI plugin routes for front-end plugin integration.	present, absent, exists, enforced

Platform configuration

Module	Description	Supported states
settings	Modify platform-wide settings including token authentication, JWT configuration, password policies, and session settings. This module has no state parameter and always applies changes.	N/A (always applies)
feature_flag	Query and manage feature flags. Only run-time flags can be modified; install-time flags are read-only.	present, absent, exists, enforced

Token and certificate management

Module	Description	Supported states
token	Create or delete OAuth2 tokens. Each run creates a new token; this module is not idempotent. The token value is only available at creation time.	present, absent
ca_certificate	Manage CA certificates for mutual TLS (mTLS) authentication.	present, absent, exists

Lookup plugin

Plugin	Description
gateway_api	Query any platform gateway API endpoint. Supports pagination, filtering, and returning objects or IDs. Use for read-only lookups of users, teams, organizations, settings, and other resources.

Additional resources

- [ansible.platform module documentation on Automation hub \(https://console.redhat.com/ansible/automation-hub/repo/published/ansible/platform/content/?showing=module\)](https://console.redhat.com/ansible/automation-hub/repo/published/ansible/platform/content/?showing=module)

1.4 Configuration as Code migration guide for Ansible Automation Platform 2.7

(New section)

If you are upgrading to Ansible Automation Platform 2.7, review the following changes to the `ansible.controller` and `ansible.hub` collections that can affect your existing Configuration as Code playbooks.

Deprecated parameters

The following parameters are deprecated and scheduled for removal in a future release of

Ansible Automation Platform. Update your playbooks to use the replacement parameters.

Module	Deprecated parameter	Replacement	Details
user	organizations	role_user_assignment module	Use the role_user_assignment module to assign organization roles to users instead of the organizations parameter.
user	is_platform_auditor	role_user_assignment module	Assign the Platform Auditor role by using the role_user_assignment module.
user	authenticators	associated_authenticators	The associated_authenticators parameter accepts a dictionary keyed by authenticator ID with values containing uid and email.
user	authenticator_uid	associated_authenticators	Use the associated_authenticators parameter instead.
role_user_assignment	object_id	object_ids	The object_ids parameter accepts a list of resource identifiers, enabling batch role assignments in a single task.

Removed modules from the ansible.hub collection

Ansible Automation Platform 2.7 removes the following modules from the ansible.hub collection.

Use the `ansible.platform` replacements instead.

Removed module	Replacement	Action required
<code>ansible.hub.ah_user</code>	<code>ansible.platform.user</code>	Update all playbooks that use <code>ansible.hub.ah_user</code> to use <code>ansible.platform.user</code> . The <code>ansible.platform.user</code> module manages users through the platform gateway and supports additional parameters such as <code>associated_authenticators</code> .
<code>ansible.hub.ah_token</code>	<code>ansible.platform.token</code>	Update all playbooks that use <code>ansible.hub.ah_token</code> to use <code>ansible.platform.token</code> . Note that the <code>ansible.platform.token</code> module is not idempotent; each run creates a new token.

New modules

The following modules are new in `ansible.platform` version 2.7:

- **feature_flag** — Query and manage feature flags. Use this module to enable or disable run-time feature flags for your platform.
- **ca_certificate** — Manage CA certificates for mutual TLS (mTLS) authentication between services.
- **role_team_assignment** — Assign roles to teams for specific resources or organizations. Supports batch operations through the `assignment_objects` parameter.
- **role_definition** — Create custom RBAC role definitions with specific permissions scoped to a content type.
- **ui_plugin_route** — Configure UI plugin routes for front-end plugin integration with the platform gateway.

New features in existing modules

The following features are available in existing modules:

- **Mutual TLS support:** The `service` and `route` modules support an `enable_mtls` parameter for mutual TLS authentication. The `enable_mtls` and `enable_gateway_auth` parameters are mutually exclusive.
- **Route timeouts:** The `service`, `route`, and `ui_plugin_route` modules support `request_timeout_seconds` and `idle_timeout_seconds` parameters for per-route timeout configuration.

- **OIDC User Identity:** The authenticator module supports OIDC User Identity configuration for platform gateway.
- **Batch role assignments:** The `role_user_assignment` module supports `object_ids` for assigning a role to a user across multiple resources in a single task.

Example: Update a playbook for 2.7

The following example shows how to update a playbook that uses deprecated parameters. Before (2.6):

```
- name: Create user with org membership
  ansible.platform.user:
    username: "demo-user"
    organizations:
      - "Demo-Organization"
    is_platform_auditor: true
```

After (2.7):

```
- name: Create user
  ansible.platform.user:
    username: "demo-user"
    state: present

- name: Assign organization role to user
  ansible.platform.role_user_assignment:
    user: "demo-user"
    role_definition: "Organization Member"
    object_ids:
      - "Demo-Organization"
    state: present

- name: Assign Platform Auditor role to user
  ansible.platform.role_user_assignment:
    user: "demo-user"
    role_definition: "Platform Auditor"
    # object_ids is not required for platform-wide roles
    state: present
```